Maze-Solver

V0.6

Generated by Doxygen 1.12.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 MazeSolver.Algorithm Enum Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Member Data Documentation . . . 5

3.1.2.1 A_STAR . . . 5

3.1.2.2 BFS . . . 5

3.1.2.3 DFS . . . 5

3.2 Maze.Dir Enum Reference . . . 6

3.2.1 Detailed Description . . . 6

3.2.2 Constructor & Destructor Documentation . . . 6

3.2.2.1 Dir() . . . 6

3.2.3 Member Function Documentation . . . 6

3.2.3.1 idx() . . . 6

3.2.4 Member Data Documentation . . . 7

3.2.4.1 EAST . . . 7

3.2.4.2 NORTH . . . 7

3.2.4.3 SOUTH . . . 7

3.2.4.4 WEST . . . 7

3.3 Maze Class Reference . . . 7

3.3.1 Detailed Description . . . 9

3.3.2 Constructor & Destructor Documentation . . . 9

3.3.2.1 Maze() . . . 9

3.3.3 Member Function Documentation . . . 10

3.3.3.1 buildNeighborTableLines() . . . 10

3.3.3.2 canMove() . . . 10

3.3.3.3 createCell() . . . 10

3.3.3.4 exportToFile() . . . 11

3.3.3.5 generateRandomMaze() [1/3] . . . 11

3.3.3.6 generateRandomMaze() [2/3] . . . 12

3.3.3.7 generateRandomMaze() [3/3] . . . 12

3.3.3.8 getGoal() . . . 13

3.3.3.9 getLength() . . . 13

3.3.3.10 getMoveCost() . . . 13

3.3.3.11 getNode() . . . 13

3.3.3.12 getStart() . . . 14

3.3.3.13 getWidth() . . . 14

3.3.3.14 importFromFile() . . . 14

3.3.3.15 isInBounds() . . . 15
3.3.3.16 isOccupied() . . . 15
3.3.3.17 setGoal() . . . 15
3.3.3.18 setStart() . . . 16
3.3.3.19 validateNeighborConnections() . . . 16
3.3.4 Member Data Documentation . . . 16
3.3.4.1 DISABLED_MOVEMENT_COST . . . 16
3.3.4.2 MAX_MOVEMENT_COST . . . 16
3.3.4.3 MIN_MOVEMENT_COST . . . 17
3.4 MazeRenderer Class Reference . . . 17
3.4.1 Detailed Description . . . 17
3.4.2 Member Function Documentation . . . 18
3.4.2.1 launchFromTitleScreen() . . . 18
3.4.2.2 resetNodeColor() . . . 18
3.4.2.3 setNodeColor() . . . 18
3.5 MazeSolver Class Reference . . . 19
3.5.1 Detailed Description . . . 19
3.5.2 Member Function Documentation . . . 20
3.5.2.1 getAlg() . . . 20
3.5.2.2 getCurrentPathCost() . . . 20
3.5.2.3 getFinalPath() . . . 20
3.5.2.4 getPathCost() . . . 20
3.5.2.5 getSolveTimeNs() . . . 21
3.5.2.6 getVisitedCount() . . . 21
3.5.2.7 isFinished() . . . 21
3.5.2.8 isSolved() . . . 21
3.5.2.9 resetSearch() . . . 21
3.5.2.10 setAlg() . . . 21
3.5.2.11 setMaze() . . . 22
3.5.2.12 updateSolve() . . . 22
3.6 MS Class Reference . . . 22
3.6.1 Detailed Description . . . 23
3.6.2 Member Function Documentation . . . 23
3.6.2.1 main() . . . 23
3.7 MazeRenderer.WeightDisplayState Enum Reference . . . 23
3.7.1 Detailed Description . . . 23
3.7.2 Member Data Documentation . . . 23
3.7.2.1 FINAL_PATH . . . 23
3.7.2.2 NORMAL . . . 24
3.7.2.3 TRAVERSED . . . 24

4 File Documentation **25**

4.1 src/Maze.java File Reference 25
4.2 src/MazeRenderer.java File Reference 25
4.3 src/MazeSolver.java File Reference 25
4.4 src/MS.java File Reference 26

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

MazeSolver.Algorithm
: Search strategies supported by this solver 5

Maze.Dir
: Cardinal directions used for indexing node neighbors 6

Maze
: Core maze model storing occupancy, neighbor links, and start/goal nodes 7

MazeRenderer
: Renderer and UI controller for the maze application 17

MazeSolver
: Incremental maze solver that supports DFS, BFS, and weighted A∗ 19

MS
: Demo application entry point 22

MazeRenderer.WeightDisplayState
: Describes how a weight label should be styled based on the solve state 23

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

src/Maze.java 25
src/MazeRenderer.java 25
src/MazeSolver.java 25
src/MS.java 26

Chapter 3

Class Documentation

3.1 MazeSolver.Algorithm Enum Reference

Search strategies supported by this solver.

Public Attributes

- DFS
- BFS
- A_STAR

3.1.1 Detailed Description

Search strategies supported by this solver.

3.1.2 Member Data Documentation

3.1.2.1 A_STAR

`MazeSolver.Algorithm.A_STAR`

3.1.2.2 BFS

`MazeSolver.Algorithm.BFS`

3.1.2.3 DFS

`MazeSolver.Algorithm.DFS`

The documentation for this enum was generated from the following file:

- src/MazeSolver.java

3.2 Maze.Dir Enum Reference

Cardinal directions used for indexing node neighbors.

Public Member Functions

- Dir (int idx)

  Creates a direction enum value with a fixed neighbor-array index.
- int idx ()

  Gets the neighbor-array index associated with this direction.

Public Attributes

- NORTH =(0)
- EAST =(1)
- SOUTH =(2)
- WEST =(3)

3.2.1 Detailed Description

Cardinal directions used for indexing node neighbors.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 Dir()

```
Maze.Dir.Dir (
            int idx)  [inline]
```

Creates a direction enum value with a fixed neighbor-array index.

Parameters

idx	index used in a node's neighbor array

3.2.3 Member Function Documentation

3.2.3.1 idx()

```
int Maze.Dir.idx ()  [inline]
```

Gets the neighbor-array index associated with this direction.

Returns

zero-based index for this direction in Node.neighbors

3.2.4 Member Data Documentation

3.2.4.1 EAST

```
Maze.Dir.EAST =(1)
```

3.2.4.2 NORTH

```
Maze.Dir.NORTH =(0)
```

3.2.4.3 SOUTH

```
Maze.Dir.SOUTH =(2)
```

3.2.4.4 WEST

```
Maze.Dir.WEST =(3)
```

The documentation for this enum was generated from the following file:

- src/Maze.java

3.3 Maze Class Reference

Core maze model storing occupancy, neighbor links, and start/goal nodes.

Classes

- enum Dir

  Cardinal directions used for indexing node neighbors.
- class **NeighborValidationResult**

  Validation result for neighbor consistency checks.
- class **Node**

  Represents a single cell in the maze grid.

Public Member Functions

- Maze (int length, int width)

  Initializes a maze with fixed dimensions.
- void createCell (int row, int col, int direction)

  Marks a cell as occupied and creates a node at that coordinate.
- void exportToFile (String filePath) throws IOException

  Export the current maze to text format for import/export.
- int getMoveCost (Node from, Node to)

  Gets the cost of moving between two adjacent nodes.
- boolean canMove (Node from, Dir direction)

  Indicates whether movement is enabled from a node in a direction.
- void setStart (int row, int col)

  Sets the start node if the given coordinate is occupied.
- void setGoal (int row, int col)

  Sets the goal node and updates the goal flag on nodes accordingly.
- Node getStart ()

  Returns the current start node.
- Node getGoal ()

  Returns the current goal node.
- Node getNode (int row, int col)

  Returns the node at a coordinate if that cell is occupied.
- boolean isInBounds (int row, int col)

  Checks whether coordinates are inside maze boundaries.
- boolean isOccupied (int row, int col)

  Checks whether a cell is a traversable/occupied cell.
- int getLength ()

  Gets the maze row count.
- int getWidth ()

  Gets the maze column count.
- NeighborValidationResult validateNeighborConnections (int maxIssuesToCollect)

  Validate all node-to-neighbor links against occupied adjacency.
- List< String > buildNeighborTableLines ()

  Build a compact, line-by-line neighbor table for occupied cells.

Static Public Member Functions

- static Maze importFromFile (String filePath) throws IOException

  Import a maze from text using this format: line 1: row count line 2: column count line 3: start: row,col line 4: end: row,col line 5+: layout rows of 1s and 0s optional: WEIGHTS followed by length ∗ width rows of N,E,S,W costs.
- static Maze generateRandomMaze (int length, int width)

  Builds a random, guaranteed-solvable maze with the given dimensions.
- static Maze generateRandomMaze (int length, int width, boolean weighted, int minWeight, int maxWeight)

  Builds a random, guaranteed-solvable maze with optional directional weights.
- static Maze generateRandomMaze (int length, int width, boolean weighted, int minWeight, int maxWeight, boolean sameCostBothDirections)

  Builds a random maze with optional weighted movement and reverse-edge matching.

Static Public Attributes

- static final int MIN_MOVEMENT_COST = 1

  Minimum allowed positive movement cost.
- static final int MAX_MOVEMENT_COST = 100

  Maximum allowed movement cost for all maps and generated mazes.
- static final int DISABLED_MOVEMENT_COST = 0

  Cost value reserved for a disabled or non-existent movement direction.

3.3.1 Detailed Description

Core maze model storing occupancy, neighbor links, and start/goal nodes.

The maze uses a rectangular grid of `Node` objects. Traversable cells contain a Node reference and blocked cells are `null`. Neighbor links represent geometric adjacency between traversable cells, while each direction has a movement cost. Enabled movement costs are positive integers from `MIN_MOVEMENT_COST` through `MAX_MOVEMENT_COST`; zero is reserved for disabled or non-existent movement. Direction order is always `[NORTH, EAST, SOUTH, WEST]`.

Example import format for `importFromFile(String)` is: row count, column count, `start: row,col`, `end: row,col`, then one layout row per maze row using `1` (path) and `0` (wall). An optional `WEIGHTS` section may follow the layout. It contains one comma-separated four-value row for every grid coordinate in row-major order. Values use `[N,E,S,W]` order; `0` means no permitted movement and positive integer values are movement costs. If the section is absent, adjacent traversable cells default to cost `1`. Example:

```
4
5
start: 3,0
end: 0,4
11111
10001
10101
11111
```

Typical implementation flow:

```
Maze maze = Maze.importFromFile("src/test_346x346.txt");
Maze.NeighborValidationResult vr = maze.validateNeighborConnections(25);
if (!vr.isValid()) {
    for (String issue : vr.getIssues()) {
        System.out.println(issue);
    }
}
maze.exportToFile("out.txt");
```

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Maze()

```
Maze.Maze (
            int length,
            int width)  [inline]
```

Initializes a maze with fixed dimensions.

A new maze starts empty: all cells are `null` (blocked) until paths are created explicitly via `createCell(int, int, int)` or populated through `importFromFile(String)`.

Parameters

length	total number of rows in the maze
width	total number of columns in the maze

3.3.3 Member Function Documentation

3.3.3.1 buildNeighborTableLines()

```
List< String > Maze.buildNeighborTableLines ()  [inline]
```

Build a compact, line-by-line neighbor table for occupied cells.

Returns

list of formatted lines describing neighbors for debugging

3.3.3.2 canMove()

```
boolean Maze.canMove (
            Node from,
            Dir direction)  [inline]
```

Indicates whether movement is enabled from a node in a direction.

Parameters

from	source node
direction	direction to inspect

Returns

true when a neighbor exists and its movement cost is positive

3.3.3.3 createCell()

```
void Maze.createCell (
            int row,
            int col,
            int direction)  [inline]
```

Marks a cell as occupied and creates a node at that coordinate.

Establishes bidirectional neighbor links immediately: this node is linked to existing occupied neighbors, and those neighbors are updated to point back. The `direction` parameter is currently reserved and ignored.

Parameters

row	row index of the cell
col	column index of the cell
direction	reserved parameter for future directional creation logic

3.3.3.4 exportToFile()

```
void Maze.exportToFile (
            String filePath) throws IOException  [inline]
```

Export the current maze to text format for import/export.

The output follows the same schema accepted by `importFromFile(String)`.

Parameters

filePath	destination file path

Exceptions

IOException	if writing fails
IllegalStateException	if start or end is not set

3.3.3.5 generateRandomMaze() [1/3]

```
static Maze Maze.generateRandomMaze (
            int length,
            int width)  [inline], [static]
```

Builds a random, guaranteed-solvable maze with the given dimensions.

The generated maze keeps the outer border blocked and carves interior corridors using a depth-first backtracking strategy to produce a classic maze appearance. For even dimensions, a follow-up pass extends corridors into the final interior row/column so right/bottom edges are not left as persistent solid strips.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions are smaller than 3

3.3.3.6 generateRandomMaze() [2/3]

```
static Maze Maze.generateRandomMaze (
            int length,
            int width,
            boolean weighted,
            int minWeight,
            int maxWeight)  [inline], [static]
```

Builds a random, guaranteed-solvable maze with optional directional weights.

When weighting is enabled, each permitted connection receives a random positive integer cost in the inclusive range `[minWeight, maxWeight]`. The five-argument overload uses the same cost in both directions by default.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)
weighted	whether to assign random movement costs
minWeight	inclusive minimum movement cost
maxWeight	inclusive maximum movement cost

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions or weight bounds are invalid

3.3.3.7 generateRandomMaze() [3/3]

```
static Maze Maze.generateRandomMaze (
            int length,
            int width,
            boolean weighted,
            int minWeight,
            int maxWeight,
            boolean sameCostBothDirections)  [inline], [static]
```

Builds a random maze with optional weighted movement and reverse-edge matching.

Parameters

length	total number of rows (minimum 3)
width	total number of columns (minimum 3)
weighted	whether to assign random movement costs
minWeight	inclusive minimum movement cost
maxWeight	inclusive maximum movement cost
sameCostBothDirections	whether each connection uses one cost in both directions

Returns

newly generated solvable maze

Exceptions

IllegalArgumentException	if dimensions or weight bounds are invalid

3.3.3.8 getGoal()

```
Node Maze.getGoal ()  [inline]
```

Returns the current goal node.

Returns

goal node, or null if not set

3.3.3.9 getLength()

```
int Maze.getLength ()  [inline]
```

Gets the maze row count.

Returns

total number of rows

3.3.3.10 getMoveCost()

```
int Maze.getMoveCost (
            Node from,
            Node to)  [inline]
```

Gets the cost of moving between two adjacent nodes.

Parameters

from	source node
to	destination node

Returns

zero when movement is disabled, otherwise the positive cost

Exceptions

IllegalArgumentException	if the nodes are not adjacent

3.3.3.11 getNode()

```
Node Maze.getNode (
            int row,
            int col)  [inline]
```

Returns the node at a coordinate if that cell is occupied.

Parameters

row	row coordinate to query
col	column coordinate to query

Returns

node at the coordinate, or null if not occupied

3.3.3.12 getStart()

```
Node Maze.getStart ()  [inline]
```

Returns the current start node.

Returns

start node, or null if not set

3.3.3.13 getWidth()

```
int Maze.getWidth ()  [inline]
```

Gets the maze column count.

Returns

total number of columns

3.3.3.14 importFromFile()

```
static Maze Maze.importFromFile (
            String filePath) throws IOException  [inline], [static]
```

Import a maze from text using this format: line 1: row count line 2: column count line 3: start: row,col line 4: end: row,col line 5+: layout rows of 1s and 0s optional: WEIGHTS followed by length $*$ width rows of N,E,S,W costs.

Cells marked `1` are created through `createCell(int, int, int)` so neighbor links are built incrementally and remain bidirectionally consistent.

Parameters

filePath	path to the maze text file

Returns

newly created Maze instance populated from file data

Exceptions

IOException	if file reading fails
IllegalArgumentException	if file contents are invalid

3.3.3.15 isInBounds()

```
boolean Maze.isInBounds (
            int row,
            int col)  [inline]
```

Checks whether coordinates are inside maze boundaries.

Parameters

row	row coordinate to test
col	column coordinate to test

Returns

true if in range, otherwise false

3.3.3.16 isOccupied()

```
boolean Maze.isOccupied (
            int row,
            int col)  [inline]
```

Checks whether a cell is a traversable/occupied cell.

Parameters

row	row coordinate to test
col	column coordinate to test

Returns

true if in bounds and occupied, otherwise false

3.3.3.17 setGoal()

```
void Maze.setGoal (
            int row,
            int col)  [inline]
```

Sets the goal node and updates the goal flag on nodes accordingly.

Parameters

row	goal row coordinate
col	goal column coordinate

3.3.3.18 setStart()

```
void Maze.setStart (
            int row,
            int col)  [inline]
```

Sets the start node if the given coordinate is occupied.

Parameters

row	start row coordinate
col	start column coordinate

3.3.3.19 validateNeighborConnections()

```
NeighborValidationResult Maze.validateNeighborConnections (
            int maxIssuesToCollect)  [inline]
```

Validate all node-to-neighbor links against occupied adjacency.

Parameters

maxIssuesToCollect	maximum number of issue messages to store.

Returns

validation result containing status, issue count, and messages

3.3.4 Member Data Documentation

3.3.4.1 DISABLED_MOVEMENT_COST

```
final int Maze.DISABLED_MOVEMENT_COST = 0  [static]
```

Cost value reserved for a disabled or non-existent movement direction.

3.3.4.2 MAX_MOVEMENT_COST

```
final int Maze.MAX_MOVEMENT_COST = 100  [static]
```

Maximum allowed movement cost for all maps and generated mazes.

3.3.4.3 MIN_MOVEMENT_COST

```
final int Maze.MIN_MOVEMENT_COST = 1  [static]
```

Minimum allowed positive movement cost.

The documentation for this class was generated from the following file:

- src/Maze.java

3.4 MazeRenderer Class Reference

Renderer and UI controller for the maze application.

Classes

- enum WeightDisplayState

  Describes how a weight label should be styled based on the solve state.

Static Public Member Functions

- static void launchFromTitleScreen ()

  Starts the renderer-driven application flow from the title screen.
- static boolean setNodeColor (Maze.Node node, Color color)

  Sets one node color using maze coordinates.
- static boolean resetNodeColor (Maze.Node node)

  Restores one node to its default semantic color.

3.4.1 Detailed Description

Renderer and UI controller for the maze application.

This class owns the full Swing-driven UX flow:

- Title screen mode selection (import or random generation)
- Maze file selection or random maze creation
- Maze grid rendering with a paint-based renderer for all maze sizes
- Runtime solver controls (start/pause/restart, delay, instant mode, map load/save/generate)
- Zoom handling (buttons and Ctrl+mouse wheel)
- Optional directional movement-cost overlay

Typical usage is through `launchFromTitleScreen()` from the app entrypoint. Example:

```
SwingUtilities.invokeLater(MazeRenderer::launchFromTitleScreen);
```

Typical extension point for custom highlighting:

```
Maze.Node node = maze.getStart();
MazeRenderer.setNodeColor(node, Color.MAGENTA);
MazeRenderer.resetNodeColor(node);
```

3.4.2 Member Function Documentation

3.4.2.1 launchFromTitleScreen()

```
static void MazeRenderer.launchFromTitleScreen ()  [inline], [static]
```

Starts the renderer-driven application flow from the title screen.

This is the recommended public entrypoint for GUI startup.

3.4.2.2 resetNodeColor()

```
static boolean MazeRenderer.resetNodeColor (
            Maze.Node node)  [inline], [static]
```

Restores one node to its default semantic color.

Default semantic color rules are: start=green, goal=blue, traversable=white, blocked=dark gray.

Parameters

node	maze node to reset

Returns

true when update was accepted

3.4.2.3 setNodeColor()

```
static boolean MazeRenderer.setNodeColor (
            Maze.Node node,
            Color color)  [inline], [static]
```

Sets one node color using maze coordinates.

Typical usage is temporary overlays (for example hints or debug markers) on top of solver coloring.

Parameters

node	maze node to color
color	target color

Returns

true when update was accepted

The documentation for this class was generated from the following file:

- src/MazeRenderer.java

3.5 MazeSolver Class Reference

Incremental maze solver that supports DFS, BFS, and weighted A∗.

Classes

- enum Algorithm

  Search strategies supported by this solver.

Public Member Functions

- void setAlg (String alg)

  Sets which algorithm to run.
- String getAlg ()

  Gets the active algorithm label.
- void setMaze (Maze maze)

  Assigns the maze instance to solve.
- Maze.Node updateSolve ()

  Advances solving by exactly one visited node.
- boolean isFinished ()

  Indicates whether this run has terminated.
- boolean isSolved ()

  Indicates whether the current run found the goal.
- int getVisitedCount ()

  Gets count of nodes visited in the current run.
- long getSolveTimeNs ()

  Gets the total computation time elapsed in the current run.
- long getPathCost ()

  Gets the total movement cost of the final path.
- long getCurrentPathCost ()

  Gets the cost of the parent route for the most recently visited node.
- List< Point > getFinalPath ()

  Gets the final path after a successful solve.
- void resetSearch ()

  Resets transient search state so solving can start fresh.

3.5.1 Detailed Description

Incremental maze solver that supports DFS, BFS, and weighted A∗.

The solver keeps internal state between calls to `updateSolve()`, so callers can visualize or inspect each step one node at a time. Reported solve time measures the elapsed time of each search-step call, excluding timer delays and UI repaint work.

Typical usage: create a solver, set algorithm and maze, then call `updateSolve()` repeatedly until `isFinished()` returns true. Example:

```
MazeSolver solver = new MazeSolver();
solver.setMaze(maze);
solver.setAlg("A*");
while (!solver.isFinished()) {
    solver.updateSolve();
}
if (solver.isSolved()) {
    List<Point> path = solver.getFinalPath();
}
```

3.5.2 Member Function Documentation

3.5.2.1 getAlg()

```
String MazeSolver.getAlg ()  [inline]
```

Gets the active algorithm label.

Returns

display label for the current algorithm

3.5.2.2 getCurrentPathCost()

```
long MazeSolver.getCurrentPathCost ()  [inline]
```

Gets the cost of the parent route for the most recently visited node.

This is the current branch cost, not the sum of all explored branches. It is updated in constant time as each node is visited and remains useful for displaying progress during an incomplete search.

Returns

current route cost, or `-1` before the first search step

3.5.2.3 getFinalPath()

```
List< Point > MazeSolver.getFinalPath ()  [inline]
```

Gets the final path after a successful solve.

The path is traced backward using the internal parent table, so the returned list begins at the goal and ends at the start.

Typical UI code can iterate as-is for reverse painting, or reverse the list first if start-to-goal order is preferred.

Returns

list of path points where `x=col, y=row`; empty if unsolved

3.5.2.4 getPathCost()

```
long MazeSolver.getPathCost ()  [inline]
```

Gets the total movement cost of the final path.

This is the sum of the directional movement costs in the path. It is `-1` before a path is found or when the search is unsuccessful.

Returns

final path cost, or `-1` when no path is available

3.5.2.5 getSolveTimeNs()

```
long MazeSolver.getSolveTimeNs ()  [inline]
```

Gets the total computation time elapsed in the current run.

Returns

elapsed computation time in nanoseconds, excluding timer and UI delays

3.5.2.6 getVisitedCount()

```
int MazeSolver.getVisitedCount ()  [inline]
```

Gets count of nodes visited in the current run.

Returns

number of visited nodes since last reset

3.5.2.7 isFinished()

```
boolean MazeSolver.isFinished ()  [inline]
```

Indicates whether this run has terminated.

Returns

true when goal was found or search space was exhausted

3.5.2.8 isSolved()

```
boolean MazeSolver.isSolved ()  [inline]
```

Indicates whether the current run found the goal.

Returns

true when goal node has been reached

3.5.2.9 resetSearch()

```
void MazeSolver.resetSearch ()  [inline]
```

Resets transient search state so solving can start fresh.

This keeps the currently assigned maze and algorithm. Typical use is after changing visualization mode or when handling a restart.

3.5.2.10 setAlg()

```
void MazeSolver.setAlg (
            String alg)  [inline]
```

Sets which algorithm to run.

Parameters

alg	algorithm label. Accepted values are DFS, BFS, A∗, ASTAR, and A_STAR (case-insensitive)

Exceptions

IllegalArgumentException	if `alg` is null or unsupported

3.5.2.11 setMaze()

```
void MazeSolver.setMaze (
            Maze maze)  [inline]
```

Assigns the maze instance to solve.

Parameters

maze	maze object to solve; start/goal are validated when solving begins

3.5.2.12 updateSolve()

```
Maze.Node MazeSolver.updateSolve ()  [inline]
```

Advances solving by exactly one visited node.

Returns

node visited in this step, or `null` if solving is complete or no work remains

Exceptions

IllegalStateException	if maze/start/goal are not configured

The documentation for this class was generated from the following file:

- src/MazeSolver.java

3.6 MS Class Reference

Demo application entry point.

Static Public Member Functions

- static void main (String[ ] args)

  Starts the maze demo.

3.6.1 Detailed Description

Demo application entry point.

Typical usage is to run this class directly from IDE or command line. It delegates all GUI and session flow to `MazeRenderer`.

3.6.2 Member Function Documentation

3.6.2.1 main()

```
static void MS.main (
            String[] args)  [inline], [static]
```

Starts the maze demo.

Delegates app flow to MazeRenderer: title screen mode selection, maze import or generation, and interactive solve UI with in-session map load/save/generate actions.

Parameters

args	command line arguments (currently unused)

The documentation for this class was generated from the following file:

- src/MS.java

3.7 MazeRenderer.WeightDisplayState Enum Reference

Describes how a weight label should be styled based on the solve state.

Public Attributes

- NORMAL
- TRAVERSED
- FINAL_PATH

3.7.1 Detailed Description

Describes how a weight label should be styled based on the solve state.

3.7.2 Member Data Documentation

3.7.2.1 FINAL_PATH

```
MazeRenderer.WeightDisplayState.FINAL_PATH
```

3.7.2.2 NORMAL

`MazeRenderer.WeightDisplayState.NORMAL`

3.7.2.3 TRAVERSED

`MazeRenderer.WeightDisplayState.TRAVERSED`

The documentation for this enum was generated from the following file:

- src/MazeRenderer.java

Chapter 4

File Documentation

4.1 src/Maze.java File Reference

```
import java.io.IOException;
```

Classes

- class Maze

  Core maze model storing occupancy, neighbor links, and start/goal nodes.
- enum Maze.Dir

  Cardinal directions used for indexing node neighbors.
- class **Maze.Node**

  Represents a single cell in the maze grid.
- class **Maze.NeighborValidationResult**

  Validation result for neighbor consistency checks.

4.2 src/MazeRenderer.java File Reference

```
import java.awt.BorderLayout;
```

Classes

- class MazeRenderer

  Renderer and UI controller for the maze application.
- enum MazeRenderer.WeightDisplayState

  Describes how a weight label should be styled based on the solve state.

4.3 src/MazeSolver.java File Reference

```
import java.awt.Point;
```

Classes

- class MazeSolver

  Incremental maze solver that supports DFS, BFS, and weighted A∗.
- enum MazeSolver.Algorithm

  Search strategies supported by this solver.

4.4 src/MS.java File Reference

```
import javax.swing.SwingUtilities;
```

Classes

- class MS

  Demo application entry point.